



Artificial Intelligence Software, HW 2

VibeCoding

ROS2



Center for
Cognitive
Modeling



Task 1

Short description: Pass the full «Beginner: CLI tools» [ROS2 tutorial](#)

Artifacts: Report in pdf format with your own screenshots.



Task 2

Short description: Configure a Dockerized ROS 2 environment to stream camera feed, detect contours, and handle interactive zoom (Z/X). Use rosdep for dependency management.

Artifacts: Dockerfile, docker-compose.yml, src/ (your ROS 2 package), and a PDF report with screenshots and usage instructions.

Requirements

- **Device Mapping:** Configure docker-compose.yml to access the host camera (e.g. /dev/video0).
- **Image Source:** Use a standard ROS 2 driver (like v4l2_camera or usb_cam) to publish images. Check with rqt.
- **Package Management:** Use rosdep to install dependencies. Do not install ROS packages via apt in Dockerfile.
- **Processing Node:** Create a node to detect contours on the video stream and publish the result on other ros topic.
- **Interaction:** Implement Zoom In (key 'Z') and Zoom Out (key 'X') functionality for the processed stream.



Task 3 (1)

Context: You have collected raw field data: High-frequency Magnetometer logs (nanosecond timestamps, noisy) and Low-frequency GPS logs (NMEA format).

Short description: Create a production-ready package to fuse these streams and visualize magnetic anomalies.

Functional Requirements:

- **Data Pipeline:** Parse heterogeneous formats, synchronize data streams by time, and filter outliers.
- **Visualization:** Generate an interactive **Plotly** heatmap.
- **UX:** Implement dynamic threshold sliders and make a possibility to add OpenStreetMap background overlay.



Task 3 (2)

Methodology:

- **TDD**: Strict Test-Driven Development. Write tests first. Coverage > 80%.
- **OOP**: Design a modular architecture (Abstract Parsers, SyncStrategy pattern).
- Use VibeCoding **MCP** for code verification and testing.

Environment:

- Use **uv**, **Pixi** or other package manager.
- Commit uv.lock or pixi.lock for absolute reproducibility.

Task 3 (3)

Artifacts: Docs in PDF format with the following sections:

- **Architecture:** UML Class diagram of your solution.
- **Metrics Analysis:** Justify your choices for synchronization windows and filtering thresholds. How do they affect the result?
- **DevEx:** Reflection on the TDD process and modern tooling (uv/pixi, MCP).

Repository: Clean structure with `src/`, `tests/`, and config files.

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521